

INF560 / Unidad 3 / GL06

[GUÍA DE LABORATORIO](#)

Modelos y Eloquent ORM

Poblar y consultar eventos-app con el ORM de Laravel

Laravel 13 · Eloquent · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 3 h

Métodos HTTP de la serie: **GET** **POST** **PUT** **DELETE**

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

En la GL05 creó la tabla `eventos` en PostgreSQL e insertó un registro de prueba usando la fachada `DB` (SQL directo). Pero el controlador de `eventos-app` sigue mostrando datos de un **arreglo estático** definido a mano, tal como quedó en la GL04. En esta guía cerramos ese círculo: crearemos el **modelo Evento**, hablaremos con la tabla mediante **Eloquent**, poblaremos la base con datos realistas y, por fin, reemplazaremos el arreglo por consultas reales.

Objetivos de la guía

- Crear el modelo `Evento` y comprender la convención modelo↔tabla de Eloquent.
- Explorar el modelo con `Tinker`: consultar, crear y leer registros como objetos.
- Generar datos de prueba con un **factory** y poblar la base con un **seeder**.
- Reemplazar el arreglo estático del controlador por `Evento::latest()->paginate()` y agregar paginación en la vista.

Requisitos previos

- La GL05 completa: tabla `eventos` migrada y conexión a PostgreSQL funcionando.
- El controlador y la vista de eventos de la GL04 (que aún usan el arreglo estático).

ALCANCE DE ESTA GUÍA

Aquí trabajamos la **lectura** y el poblado de datos. El CRUD completo —crear, editar y borrar eventos desde formularios— es el tema de la **GL07**, donde usaremos un `resource controller`. Por ahora nos concentramos en el modelo, los datos y la consulta con paginación.

01 Teoría breve: Eloquent, el ORM de Laravel

1.1 ¿Qué es un ORM?

Un **ORM** (Object-Relational Mapping) traduce entre dos mundos: las **filas** de una tabla y los **objetos** de PHP. En lugar de escribir SQL a mano, usted trabaja con objetos: una fila de `eventos` se convierte en un objeto `Evento`, y sus columnas se vuelven propiedades (`$evento->titulo`). **Eloquent** es el ORM que trae Laravel.

1.2 Convención sobre configuración

Eloquent asume convenciones para ahorrarle configuración. Un modelo `Evento` (singular, en PascalCase) se asocia por defecto a la tabla `eventos` (plural). La llave primaria se llama `id`, y Eloquent gestiona solo las columnas `created_at` y `updated_at`. Cuando trabajamos con nombres en español conviene declarar la tabla de forma explícita, porque el pluralizador de Laravel razona en inglés y a veces no acierta.

1.3 ¿Por qué usar Eloquent y no SQL directo?

- Menos código repetitivo: leer, crear o filtrar registros toma una línea.
- Más seguro: Eloquent usa consultas preparadas, lo que previene la inyección SQL.
- Más expresivo: `Evento::where('cupo', '>', 100)->get()` se lee casi como una frase.

DEFINICIÓN — MODELO ELOQUENT

Un **modelo** es una clase PHP que representa una tabla. Cada instancia del modelo es una fila; una consulta puede devolver una **colección** (varias filas). Los modelos viven en `app/Models/`.

02 Ejercicio 1 — Crear el modelo Evento

Paso 1 — Generar el modelo

```
terminal Copiar

$ php artisan make:model Evento

INFO  Model [app/Models/Evento.php] created successfully.
```

Se crea el archivo `app/Models/Evento.php` con una clase vacía. Vamos a completarlo.

Paso 2 — Configurar el modelo

Abra el modelo y déjelo así:

```
app/Models/Evento.php Copiar

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Evento extends Model
{
    use HasFactory;

    // La tabla asociada (explícita por trabajar en español)
    protected $table = 'eventos';

    // Columnas que se pueden asignar de forma masiva
    protected $fillable = [
        'titulo', 'slug', 'descripcion', 'fecha',
        'lugar', 'cupos', 'precio', 'imagen', 'publicado',
    ];

    // Conversión automática de tipos al leer/escribir
    protected $casts = [
        'fecha' => 'datetime',
        'precio' => 'decimal:2',
        'publicado' => 'boolean',
    ];
}
```

Tres piezas importan aquí:

- `use HasFactory` habilita `Evento::factory()`, que usaremos para generar datos en el Ejercicio 3.
- `$fillable` lista las columnas que se pueden llenar con `Evento::create([...])`. Protege contra la **asignación masiva** (que un formulario malicioso escriba campos que no debería).
- `$casts` convierte tipos automáticamente: `fecha` llega como objeto `Carbon` (fechas), `publicado` como `true/false` y `precio` con dos decimales.

SIN \$FILLABLE, NADA SE GUARDA

Si intenta `Evento::create([...])` con un campo que no está en `$fillable`, Laravel lanza `Add [campo] to fillable property`. Es una protección, no un error: agregue la columna a la lista.

03 Ejercicio 2 — Explorar el modelo con Tinker

Tinker es una consola interactiva sobre su aplicación: puede ejecutar código de Laravel línea por línea. Es la mejor forma de conocer Eloquent. Ábrala con `php artisan tinker`.

Consultar los registros existentes

```
php artisan tinker Copiar

> Evento::count()
= 1

> Evento::all();
= Illuminate\Database\Eloquent\Collection {
  all: [
    App\Models\Evento {
      id: 1,
      titulo: "Concierto de aniversario UATF",
      lugar: "Paraninfo Universitario",
      ...
    },
  ],
}
```

Ese registro es el que insertó en la GL05. Note la diferencia con la GL05: antes recibía datos crudos; ahora recibe **objetos Evento** dentro de una **colección**.

Leer una fila y acceder a sus propiedades

```
php artisan tinker Copiar

> $e = Evento::find(1);      // buscar por id

> $e->titulo
= "Concierto de aniversario UATF"

> $e->fecha                  // gracias a $casts, es un objeto Carbon
= Illuminate\Support\Carbon @1793044800 { date: 2026-11-05 20:00:00 }

> $e->fecha->format('d/m/Y')
= "05/11/2026"
```

Como `fecha` está casteada a `datetime`, puede formatearla con métodos de Carbon sin conversiones manuales.

Crear un registro con Eloquent

```
php artisan tinker Copiar

> Evento::create([
  'titulo'      => 'Feria de Ciencias 2026',
  'slug'        => 'feria-de-ciencias-2026',
  'descripcion' => 'Proyectos de estudiantes de la facultad.',
  'fecha'       => '2026-10-20 09:00:00',
  'lugar'       => 'Campus Central',
  'cupos'       => 500,
  'precio'      => 0,
]);
= App\Models\Evento { id: 2, titulo: "Feria de Ciencias 2026", ... }
```

COMPARE CON LA GL05

En la GL05 insertó con `DB::table('eventos')->insert([...])` y tuvo que poner `created_at` y `updated_at` a mano. Con `Evento::create([...])` Eloquent los llena solo y le devuelve el objeto ya guardado. Ese es el valor del ORM.

Filtrar con el constructor de consultas

php artisan tinker

Copiar

```
> Evento::where('cupo', '>', 100)->get();    // eventos con más de 100 cupos
> Evento::orderBy('fecha')->first();        // el próximo por fecha
> Evento::pluck('titulo');                  // solo los títulos
```

NOTA

Para salir de Tinker escriba `exit`. Si edita un modelo, cierre y vuelva a abrir Tinker: la sesión no recarga los cambios automáticamente.

04 Ejercicio 3 — Poblar la base con un Factory

Insertar eventos a mano no escala. Un **factory** es una plantilla que genera registros con datos falsos pero realistas (usando la librería `Faker`), ideal para llenar la base durante el desarrollo.

Paso 1 — Generar el factory

terminal

Copiar

```
$ php artisan make:factory EventoFactory
```

```
INFO  Factory [database/factories/EventoFactory.php] created successfully.
```

Paso 2 — Definir los datos falsos

Complete el método `definition()`, que devuelve un arreglo con un valor para cada columna:

database/factories/EventoFactory.php

Copiar

```
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;

class EventoFactory extends Factory
{
    public function definition(): array
    {
        $titulo = rtrim(fake()->sentence(3), '.');

        return [
            'titulo'      => $titulo,
            'slug'        => Str::slug($titulo) . '-' . fake()->unique()->numberBetween(1,
99999),
            'descripcion' => fake()->paragraph(),
            'fecha'       => fake()->dateTimeBetween('now', '+3 months'),
            'lugar'       => fake()->randomElement(['Parainfo', 'Coliseo Universitario',
'Aula Magna']),
```

```

        'cupo'      => fake()->numberBetween(50, 500),
        'precio'    => fake()->randomElement([0, 15, 25, 40, 60]),
        'publicado' => fake()->boolean(80),    // 80% de probabilidad de true
    ];
}
}

```

Puntos clave del factory:

- `fake()` es Faker: `sentence()`, `paragraph()`, `numberBetween()` y `randomElement()` generan valores variados en cada corrida.
- `slug` debe ser único (la columna tiene esa restricción); por eso añadimos `fake()->unique()->numberBetween(...)` al final.
- No incluimos `imagen` porque es `nullable`; quedará en `null`.

¿CÓMO SE ENLAZA EL FACTORY CON EL MODELO?

Por convención: `EventoFactory` ↔ modelo `Evento`. Como el modelo tiene `use HasFactory`, ya puede llamar `Evento::factory()`. Pruébalo en Tinker: `Evento::factory(3)->create()` crea tres eventos al instante.

05 Ejercicio 4 — Poblar de forma repetible con un Seeder

Un **seeder** ejecuta el poblado de forma ordenada y versionada en Git, para que todo el equipo obtenga los mismos datos de arranque con un comando.

Paso 1 — Generar el seeder

```

terminal
$ php artisan make:seeder EventoSeeder

```

Paso 2 — Usar el factory dentro del seeder

```

database/seeder/EventoSeeder.php

<?php

namespace Database\Seeders;

use App\Models\Evento;
use Illuminate\Database\Seeder;

class EventoSeeder extends Seeder
{
    public function run(): void
    {
        Evento::factory()->count(20)->create();
    }
}

```

Paso 3 — Registrar el seeder

Laravel ejecuta `DatabaseSeeder`. Llame desde ahí a su seeder:

```

database/seeder/DatabaseSeeder.php

public function run(): void
{

```

```
$this->call([
    EventoSeeder::class,
]);
}
```

Paso 4 — Ejecutar y verificar

Recree las tablas y puebla en un solo paso, o solo ejecute los seeders:

```
terminal Copiar

$ php artisan migrate:fresh --seed # recrea tablas y puebla (¡borra datos!)
# o, sin recrear:
$ php artisan db:seed

$ php artisan tinker
> Evento::count()
= 20
```

NOTA

Ahora, dentro de `psql`, el comando `\dt` ya lista la tabla `eventos` con contenido, y `SELECT count(*) FROM eventos;` devuelve 20. La base dejó de estar vacía.

06 Ejercicio 5 — Reemplazar el arreglo por Eloquent

Este es el objetivo mayor de la guía. Hasta ahora el controlador entregaba a la vista un arreglo escrito a mano. Vamos a cambiarlo por una consulta real a la base, con paginación.

Antes — el controlador con el arreglo estático (GL04)

```
app/Http/Controllers/EventoController.php Copiar

public function index()
{
    $eventos = [
        ['titulo' => 'Concierto...', 'lugar' => 'Paraninfo', ...],
        ['titulo' => 'Feria...', 'lugar' => 'Campus', ...],
        // ... datos escritos a mano
    ];

    return view('eventos.index', compact('eventos'));
}
```

Después — consultando con Eloquent

```
app/Http/Controllers/EventoController.php Copiar

use App\Models\Evento;

public function index()
{
    // latest() ordena por created_at desc; paginate divide en páginas de 9
    $eventos = Evento::latest()->paginate(9);

    return view('eventos.index', compact('eventos'));
}
```

No olvide el `use App\Models\Evento;` al inicio del controlador. `latest()` es un atajo de `orderBy('created_at', 'desc')`, y `paginate(9)` trae los eventos de a nueve por página.

Actualizar la vista: de arreglo a objeto

Antes, cada evento era un arreglo asociativo y se accedía con corchetes (`$evento['titulo']`). Ahora es un objeto y se accede con flecha (`$evento->titulo`). Ajuste el bucle de la vista:

```
resources/views/eventos/index.blade.php Copiar

<div class="grid grid-cols-1 md:grid-cols-3 gap-6">
  @foreach ($eventos as $evento)
    <article class="rounded-xl border p-4">
      <h3 class="font-semibold">{{ $evento->titulo }}</h3>
      <p class="text-sm text-gray-500">{{ $evento->lugar }}</p>
      <p class="text-sm">{{ $evento->fecha->format('d/m/Y') }}</p>
    </article>
  @endforeach
</div>

{{-- Enlaces de paginación (Tailwind) --}}
<div class="mt-6">
  {{ $eventos->links() }}
</div>
```

Como `paginate()` devuelve un paginador, `$eventos->links()` dibuja los controles «anterior / siguiente» ya con estilos de Tailwind. Recargue la página: verá los 20 eventos del seeder repartidos en páginas de nueve.

LO QUE ACABA DE LOGRAR

La lista de eventos ya no vive en el código: sale de PostgreSQL a través de Eloquent, ordenada y paginada. A partir de aquí, cualquier evento que agregue a la base aparece solo en la aplicación. En la **GL07** daremos el siguiente paso: crear, editar y borrar eventos desde formularios (CRUD completo).

07 Trabajo independiente

Repita el flujo modelo → factory → seeder con la tabla `categorias` que creó en el trabajo independiente de la GL05.

Consigna

- Cree el modelo `Categoria` con `$table = 'categorias'`, su `$fillable` (`nombre`, `slug`, `descripcion`) y use `HasFactory`.
- Cree `CategoriaFactory` con datos realistas y `slug` único.
- Cree `CategoriaSeeder`, regístrelo en `DatabaseSeeder` y puebla 6 categorías.
- Verifique con `Categoria::count()` en Tinker.

Entregables

- Commit atómico, por ejemplo: `git commit -m "feat: modelo, factory y seeder de Categoria"`.
- Push a `INF560-2026-02-Apellido` y captura del conteo de categorías.

08 Checklist de verificación

- ☐ El modelo `Evento` tiene `$table`, `$fillable` y `$casts` correctos y use `HasFactory`.
- ☐ Exploró el modelo en Tinker: `count`, `find`, `create` y una consulta con `where`.
- ☐ El factory `EventoFactory` genera datos realistas con `slug` único.
- ☐ El seeder `EventoSeeder` está registrado en `DatabaseSeeder` y se ejecutó.

- ☐ Evento::count() devuelve 20 (o el número que sembró).
- ☐ El controlador usa Evento::latest()->paginate(9) en vez del arreglo.
- ☐ La vista accede con \$evento->propiedad y muestra \$eventos->links().
- ☐ Trabajo independiente: modelo, factory y seeder de Categoría en GitHub.

09 Problemas frecuentes

Síntoma	Causa y solución
Add [titulo] to fillable property	El campo no está en \$fillable del modelo. Agréguelo a la lista.
Class "...EventoFactory" not found	Falta use HasFactory en el modelo o el factory está mal nombrado (debe ser EventoFactory).
Call to undefined method ...factory()	El modelo no usa el trait HasFactory. Agregue use HasFactory;.
relation "eventos" does not exist	No corrió las migraciones de la GL05. Ejecute php artisan migrate.
Class "Str" not found	Falta el import use Illuminate\Support\Str; en el factory.
Unique violation: slug	El factory repitió un slug. Use fake()->unique() como en el ejemplo.
Los cambios del modelo no aparecen en Tinker	Tinker no recarga solo. Escriba exit y vuelva a abrir php artisan tinker.

10 Comandos de referencia

comandos usados en esta guía	Copiar
<pre># Generar modelo, factory y seeder php artisan make:model Evento php artisan make:factory EventoFactory php artisan make:seeder EventoSeeder # Poblar la base php artisan db:seed # ejecuta los seeders php artisan migrate:fresh --seed # recrea tablas y puebla (¡borra datos!) # Explorar con Tinker php artisan tinker > Evento::count() > Evento::all() > Evento::find(1) > Evento::create([...]) > Evento::factory(5)->create() > Evento::where('cupo', '>', 100)->get() > exit</pre>	

11 Rúbrica de evaluación

Criterio	Pts
Modelo Evento con \$table, \$fillable y \$casts correctos	15
Exploración del modelo en Tinker (con evidencia)	10

Criterio	Pts
Factory EventoFactory con datos realistas y slug único	20
Seeder registrado y ejecutado (base poblada con 20 eventos)	15
Controlador refactorizado a Eloquent con paginación	20
Vista actualizada a acceso por objeto + enlaces de paginación	10
Trabajo independiente: modelo, factory y seeder de Categoría	10
Total	100